

# SGT UNIVERSITY

School of Engineering and Technology

Department of CSE

## DATABASE MANAGEMENT SYSTEMS LABORATORY

### Experiment 3

## SQL Joins — NATURAL, INNER, and OUTER

| Field     | Details                                                     |
|-----------|-------------------------------------------------------------|
| Student   | Harshit Khemani                                             |
| Roll No.  | 241302081                                                   |
| Programme | B.Tech CSE (AI/ML)                                          |
| Section   | Section - C                                                 |
| Lab Date  | 02 September 2026                                           |
| Dataset   | departments (6 rows) + employees (7 rows) linked by deptID  |
| Methods   | NATURAL JOIN, INNER JOIN, LEFT OUTER JOIN, RIGHT OUTER JOIN |

# 1. Aim

To combine rows from two related tables — departments and employees — using NATURAL JOIN, INNER JOIN, LEFT OUTER JOIN, and RIGHT OUTER JOIN, and to observe how each join type handles unmatched rows.

## 2. Theory

### 2.1 NATURAL JOIN

A NATURAL JOIN automatically matches rows on every column that appears in both tables with the same name. Here, both tables share deptID, so the join condition is implicit. Only rows with a matching deptID in both tables are returned.

### 2.2 INNER JOIN

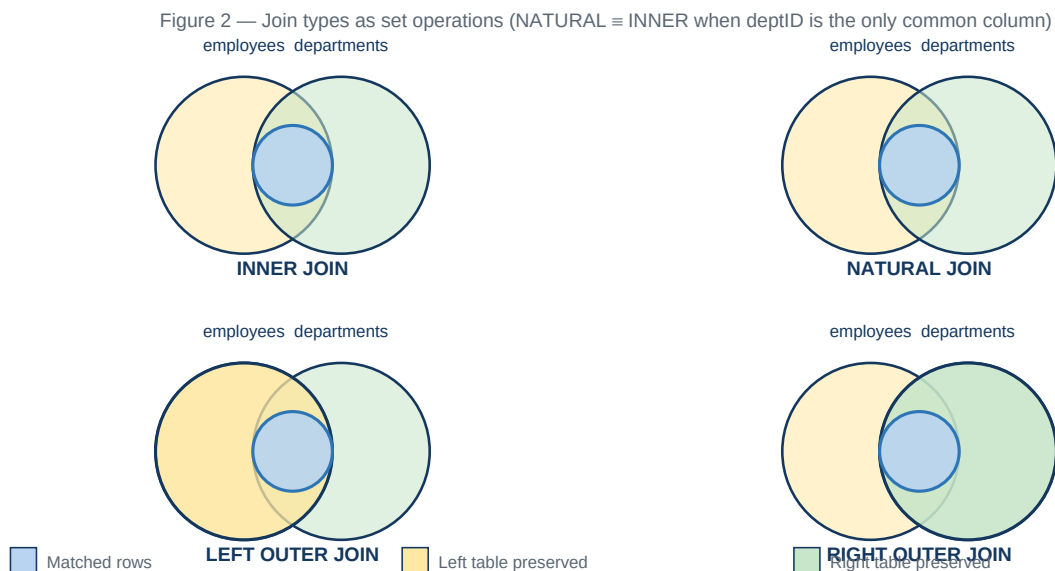
An INNER JOIN returns only the rows where the join condition is satisfied. INNER JOIN departments ON employees.deptID = departments.deptID produces the same result as NATURAL JOIN when deptID is the sole common column.

### 2.3 LEFT OUTER JOIN

A LEFT OUTER JOIN keeps every row from the left table. If no match exists in the right table, the right-hand columns are filled with NULL. Employee Neha Gupta (deptID 99) has no matching department and therefore appears with a null department name.

### 2.4 RIGHT OUTER JOIN

A RIGHT OUTER JOIN keeps every row from the right table. Unmatched left-hand columns become NULL. School of Humanities (deptID 6) has no employees and therefore appears with null employee fields.



## 3. Schema

Two tables share the column deptID. The diagram below shows how they relate before any join is applied.

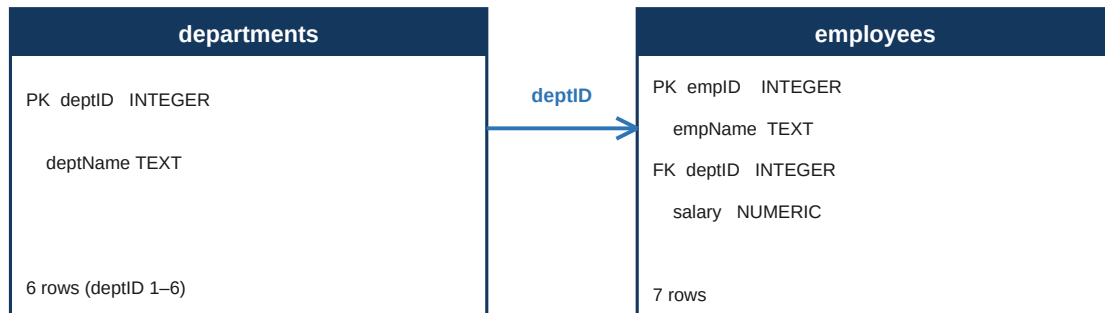


Figure 1 — Schema: employees.deptID references departments.deptID

| Table       | Attribute | Type           | Role                   |
|-------------|-----------|----------------|------------------------|
| departments | deptID    | INTEGER / INT  | Primary key            |
| departments | deptName  | TEXT / VARCHAR | School name            |
| employees   | empID     | INTEGER / INT  | Primary key            |
| employees   | empName   | TEXT / VARCHAR | Employee name          |
| employees   | deptID    | INTEGER / INT  | Join key → departments |
| employees   | salary    | NUMERIC(10,2)  | Monthly salary         |

## 4. Procedure

1. Create departments and employees with a shared deptID column.
2. Insert six departments and seven employees, including deliberate mismatches for outer joins.
3. Display both base tables.
4. Run NATURAL JOIN and INNER JOIN — expect six matched rows.
5. Run LEFT OUTER JOIN — expect seven rows (one employee without a department).
6. Run RIGHT OUTER JOIN — expect seven rows (one department without employees).

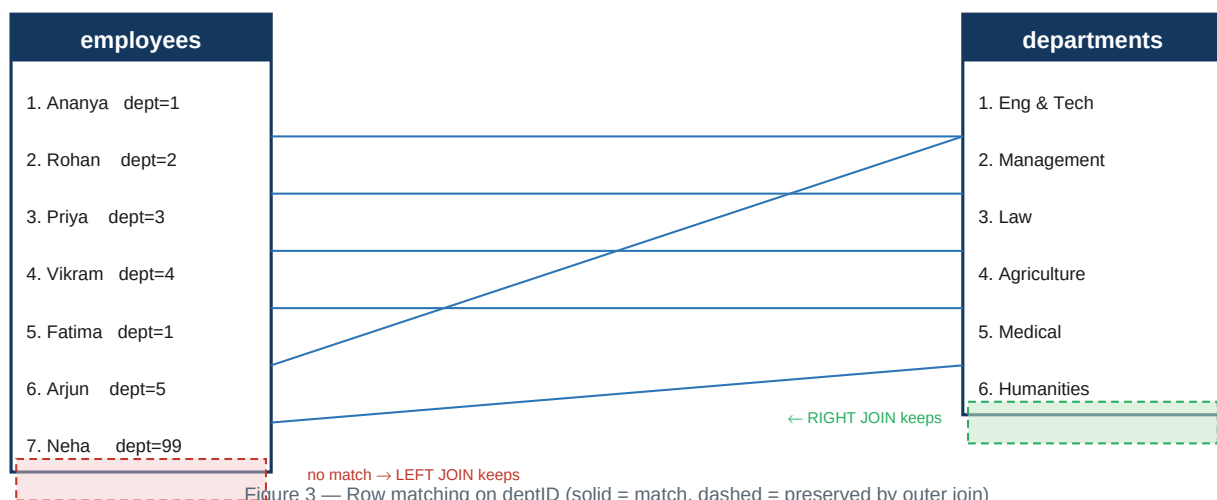


Figure 3 — Row matching on deptID (solid = match, dashed = preserved by outer join)

## 5. Source Code

```

-- =====
-- DBMS Lab · 02-09-2026
-- SQL Joins: NATURAL, INNER, LEFT OUTER, RIGHT OUTER
-- SQLite (sql.js compiler / DB Browser / sqlite3)
-- For the class SQL Server, run joins.sqlserver.sql instead.
-- =====

DROP TABLE IF EXISTS employees;
DROP TABLE IF EXISTS departments;

-- -----
-- 1. Create related tables (common column: deptID)
-- -----
CREATE TABLE departments (
    deptID INTEGER PRIMARY KEY,
    deptName TEXT NOT NULL
);

CREATE TABLE employees (
    empID INTEGER PRIMARY KEY,
    empName TEXT NOT NULL,
    deptID INTEGER NOT NULL,
    salary NUMERIC(10, 2) NOT NULL CHECK (salary > 0)
);

-- -----
-- 2. Insert sample rows
--   • deptID 6 (Humanities) has no employees → visible in RIGHT OUTER JOIN
--   • empID 7 points to deptID 99 (missing) → visible in LEFT OUTER JOIN
-- -----
INSERT INTO departments (deptID, deptName) VALUES
(1, 'School of Engineering and Technology'),
(2, 'School of Management'),
(3, 'School of Law'),
(4, 'School of Agricultural Sciences'),
(5, 'School of Medical Sciences'),
(6, 'School of Humanities');

INSERT INTO employees (empID, empName, deptID, salary) VALUES
(1, 'Ananya Sharma', 1, 92000.00),
(2, 'Rohan Mehta', 2, 78000.00),
(3, 'Priya Nair', 3, 85000.00),
(4, 'Vikram Singh', 4, 76000.00),
(5, 'Fatima Khan', 1, 88000.00),
(6, 'Arjun Patel', 5, 95000.00),
(7, 'Neha Gupta', 99, 54000.00);

-- -----
-- 3. Base tables (reference)
-- -----
SELECT * FROM departments;
SELECT * FROM employees;

-- -----
-- 4. NATURAL JOIN
--   Matches rows on every column with the same name (here: deptID).
--   Only rows with a matching deptID in BOTH tables appear.
-- -----
SELECT empID, empName, deptName, salary
FROM employees
NATURAL JOIN departments;

-- -----
-- 5. INNER JOIN
--   Explicit join condition; same result as NATURAL JOIN here
--   because deptID is the only common column.
-- -----
SELECT e.empID, e.empName, d.deptName, e.salary
FROM employees AS e
INNER JOIN departments AS d ON e.deptID = d.deptID;

-- -----
-- 6a. LEFT OUTER JOIN
--   All rows from the LEFT table (employees); unmatched dept
--   columns are NULL (empID 7 → deptID 99 has no department).
-- -----
SELECT e.empID, e.empName, e.deptID, d.deptName, e.salary
FROM employees AS e
LEFT OUTER JOIN departments AS d ON e.deptID = d.deptID;

-- -----
-- 6b. RIGHT OUTER JOIN
--   All rows from the RIGHT table (departments); unmatched
--   employee columns are NULL (deptID 6 has no employees).
--   Requires SQLite 3.39+; see joins.sqlserver.sql for T-SQL.
-- -----
SELECT e.empID, e.empName, d.deptID, d.deptName, e.salary
FROM employees AS e

```

```
RIGHT OUTER JOIN departments AS d ON e.deptID = d.deptID;
```

## 6. Output

The script was executed in SQLite. Each query's result set is shown below in execution order.

```
SQL> DROP TABLE IF EXISTS employees;
-- OK

SQL> DROP TABLE IF EXISTS departments;
-- OK

SQL> CREATE TABLE departments (
    deptID    INTEGER PRIMARY KEY,
    deptName  TEXT NOT NULL
);
-- OK

SQL> CREATE TABLE employees (
    empID     INTEGER PRIMARY KEY,
    empName   TEXT NOT NULL,
    deptID    INTEGER NOT NULL,
    salary    NUMERIC(10, 2) NOT NULL CHECK (salary > 0)
);
-- OK

SQL> INSERT INTO departments (deptID, deptName) VALUES
    (1, 'School of Engineering and Technology'),
    (2, 'School of Management'),
    (3, 'School of Law'),
    (4, 'School of Agricultural Sciences'),
    (5, 'School of Medical Sciences'),
    (6, 'School of Humanities');
-- 6 row(s) inserted

SQL> INSERT INTO employees (empID, empName, deptID, salary) VALUES
    (1, 'Ananya Sharma', 1, 92000.00),
    (2, 'Rohan Mehta', 2, 78000.00),
    (3, 'Priya Nair', 3, 85000.00),
    (4, 'Vikram Singh', 4, 76000.00),
    (5, 'Fatima Khan', 1, 88000.00),
    (6, 'Arjun Patel', 5, 95000.00),
    (7, 'Neha Gupta', 99, 54000.00);
-- 13 row(s) inserted

SQL> SELECT * FROM departments;
deptID  deptName
-----
1       School of Engineering and Technology
2       School of Management
3       School of Law
4       School of Agricultural Sciences
5       School of Medical Sciences
6       School of Humanities

SQL> SELECT * FROM employees;
empID  empName    deptID  salary
-----
1      Ananya Sharma    1      92000
2      Rohan Mehta      2      78000
3      Priya Nair       3      85000
4      Vikram Singh     4      76000
5      Fatima Khan      1      88000
6      Arjun Patel      5      95000
7      Neha Gupta       99     54000

SQL> SELECT empID, empName, deptName, salary
FROM employees
NATURAL JOIN departments;
empID  empName    deptName                                salary
-----
1      Ananya Sharma    School of Engineering and Technology    92000
2      Rohan Mehta      School of Management                    78000
3      Priya Nair       School of Law                           85000
4      Vikram Singh     School of Agricultural Sciences          76000
5      Fatima Khan      School of Engineering and Technology    88000
6      Arjun Patel      School of Medical Sciences              95000

SQL> SELECT e.empID, e.empName, d.deptName, e.salary
FROM employees AS e
INNER JOIN departments AS d ON e.deptID = d.deptID;
empID  empName    deptName                                salary
-----
1      Ananya Sharma    School of Engineering and Technology    92000
2      Rohan Mehta      School of Management                    78000
3      Priya Nair       School of Law                           85000
4      Vikram Singh     School of Agricultural Sciences          76000
5      Fatima Khan      School of Engineering and Technology    88000
```

```

6      Arjun Patel      School of Medical Sciences      95000

SQL> SELECT e.empID, e.empName, e.deptID, d.deptName, e.salary
FROM employees AS e
LEFT OUTER JOIN departments AS d ON e.deptID = d.deptID;
empID  empName      deptID  deptName      salary
-----
1      Ananya Sharma  1       School of Engineering and Technology  92000
2      Rohan Mehta   2       School of Management                 78000
3      Priya Nair    3       School of Law                       85000
4      Vikram Singh  4       School of Agricultural Sciences      76000
5      Fatima Khan   1       School of Engineering and Technology  88000
6      Arjun Patel   5       School of Medical Sciences           95000
7      Neha Gupta    99
54000

SQL> SELECT e.empID, e.empName, d.deptID, d.deptName, e.salary
FROM employees AS e
RIGHT OUTER JOIN departments AS d ON e.deptID = d.deptID;
empID  empName      deptID  deptName      salary
-----
1      Ananya Sharma  1       School of Engineering and Technology  92000
2      Rohan Mehta   2       School of Management                 78000
3      Priya Nair    3       School of Law                       85000
4      Vikram Singh  4       School of Agricultural Sciences      76000
5      Fatima Khan   1       School of Engineering and Technology  88000
6      Arjun Patel   5       School of Medical Sciences           95000
7      Neha Gupta    6       School of Humanities

```

## 7. Results



Figure 4 — Result set sizes from this lab dataset

| Join type        | Rows returned | Unmatched handling                                  |
|------------------|---------------|-----------------------------------------------------|
| NATURAL JOIN     | 6             | Drops non-matching rows from both sides             |
| INNER JOIN       | 6             | Same as NATURAL JOIN here                           |
| LEFT OUTER JOIN  | 7             | Keeps Neha Gupta (deptID 99); deptName is NULL      |
| RIGHT OUTER JOIN | 7             | Keeps Humanities (deptID 6); employee cols are NULL |

INNER JOIN and NATURAL JOIN return only the six employees whose deptID exists in departments. LEFT OUTER JOIN adds the orphan employee; RIGHT OUTER JOIN adds the department with no staff. Together they illustrate how outer joins preserve rows that inner joins would discard.

## 8. Conclusion

Joins combine related tables on a common key. INNER JOIN and NATURAL JOIN return only matching pairs. Outer joins extend one side: LEFT OUTER JOIN keeps all employees, RIGHT OUTER JOIN keeps all departments. Choosing the correct join depends on whether unmatched rows from the left, right, or neither side must appear in the result.